FRONTEND · BIMESTRE 02

Navegadores Web e API

Aula 04. Refinar a forma como o frontend e backend se comunicam — entendendo o navegador como ambiente de execução.

De onde chegamos até aqui

O frontend passou a conversar de verdade com o backend via Axios — fechando o ciclo de requisição e resposta. Agora o objetivo muda: refinar a forma como as duas camadas se comunicam, se protegem e se apoiam na própria estrutura do navegador.

Para isso, precisamos entender o navegador como um programa complexo — com memória própria, ferramentas de inspeção e mecanismos de armazenamento.

O que é um navegador

Depois do próprio sistema operacional, é o programa mais complexo que um usuário comum roda no seu computador ou celular.

Dentro dele convivem:

- **Motor de renderização** — transforma HTML/CSS em pixels na tela
- **Motor de JavaScript** — interpreta e executa JS
- **Pilha de rede** — fala HTTP, TLS, DNS
- **Mecanismos de armazenamento** — cookies, localStorage, sessionStorage, cache, IndexedDB

`Cada aba é isolada em seu próprio processo — uma aba travando não derruba o navegador inteiro.`

`window` — o objeto global do frontend

console.js

```
// document é uma propriedade de window
→ window.document === document
true
// window guarda outras informações do ambiente
→ window.innerWidth, window.innerHeight
# dimensões da viewport
→ window.location.href
# URL atual
```

Tudo que parecia "solto" (document, console, localStorage, fetch, alert) é propriedade ou método de `window`.

Window também dispara eventos

- **resize** — quando a janela muda de tamanho (novo width/height)
- **scroll** — quando a página é rolada (novo scrollY/scrollX)

```
window.addEventListener("resize", () => {
  console.log(window.innerWidth);
});

window.addEventListener("scroll", () => {
  console.log(window.scrollY);
});
```

Eventos de window — pertencem à janela como um todo, não a elementos na árvore DOM.

DevTools — a memória do navegador exposta

Abre com `Ctrl+Shift+I`. Três abas principais:

- → **Elements** — DOM em memória, alterações feitas por JS, painel de estilos renderizados (editável em tempo real)
- → **Console** — exibir logs e executar JS ao vivo contra o window/document real
- → **Network** — toda requisição HTTP em cascata (waterfall), com timing detalhado

---

O Console é uma porta de entrada viva para o ambiente de execução — muito mais poderoso que apenas ler logs.

Por que as requisições aparecem em cascata

O navegador não sabe, de antemão, todos os arquivos que vai precisar. Descobre conforme processa a resposta:

1. Faz requisição pelo HTML
2. Parseia HTML, construindo o DOM linha por linha
3. Encontra tags que referenciam outros arquivos (`<link>`, `<script>`, `<img>`)
4. Dispara requisição para cada asset naquele momento
5. Cada asset pode referenciar outros (CSS importa fonte, JS chama API)

Essa descoberta progressiva forma a "cascata" visível na aba Network.

Cookies — memória para HTTP stateless

HTTP é stateless: cada requisição chega sem memória da anterior. Cookies resolvem isso — pequenos pedaços de dado (chave=valor) que o navegador reenvia automaticamente em toda requisição para aquele domínio.

É assim que um site "lembra" que você está logado entre uma página e outra.

Lendo e escrevendo cookies

console.js

```
// LER: retorna todos os cookies como string
→ document.cookie
"tema=escuro; idioma=pt-BR"
// ESCREVER: adiciona/atualiza cookie
→ document.cookie = "tema=escuro; path=/; max-age=3600"
```

`document.cookie` é acessado via `window.document.cookie` — fixando a hierarquia da seção anterior.

Headers HTTP: Cookie e Set-Cookie

Requisição (frontend → backend)

```
GET /usuarios/perfil HTTP/1.1
Host: api.meusite.com
Cookie: sessionId=a1b2c3; tema=escuro
```

Resposta (backend → frontend)

```
HTTP/1.1 200 OK
Set-Cookie: sessionId=a1b2c3; HttpOnly; Secure; SameSite=Strict; Max-Age=3600
Set-Cookie: tema=escuro; Path=/
```

O navegador automaticamente monta/atualiza o cookie — o frontend não precisa rodar nenhum código.

Max-Age, HttpOnly e Secure

- **Max-Age** — em segundos, por quanto tempo o cookie continua válido antes de ser descartado automaticamente
- **HttpOnly** — torna invisível para `document.cookie`; JS não consegue ler nem escrever. Protege o cookie de XSS
- **Secure** — cookie só é enviado em conexões HTTPS, protegendo de captura em trânsito

Juntos, esses três definem: por quanto tempo, quem pode acessar, e em que condições o cookie trafega.

Qualquer cookie de autenticação em produção tem esses três.

Cookies no frontend — Axios envia automático

Frontend em localhost:5173. **Mesmo domínio (localhost:3000)** — navegador já envia o cookie automaticamente:

```
// Requisição pra localhost:3000
axios.post("http://localhost:3000/usuarios", { nome: "Ana" });
→ cookie vai automaticamente no header!
```

Domínios diferentes (porta 8080) — precisa liberar CORS E falar ao Axios:

```
// Requisição pra localhost:8080 (porta diferente!)
axios.post("http://localhost:8080/usuarios", { nome: "Ana" }, {
  withCredentials: true ← permite enviar cookie mesmo com CORS
});
Backend precisa responder com:
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Credentials: true
```

Sem `withCredentials`, cookie não viaja entre portas. Com ele + CORS liberado, cookie vai normalmente.

LocalStorage e SessionStorage

Vistos na aba Application do DevTools — outros mecanismos de armazenamento:

- **localStorage** — persiste indefinidamente até ser apagado manualmente
- **sessionStorage** — dura só a sessão daquela aba

Diferença crítica: **nenhum dos dois é enviado automaticamente ao backend como o cookie** — ficam só no navegador, a menos que o próprio JS os leia e mande manualmente.

JWT — JSON Web Token

Autenticação stateless: em vez do backend guardar sessão em memória/banco, emite um token auto-contido, assinado, com dados do usuário dentro.

Estrutura: `header.payload.signature` — três partes separadas por ponto, cada uma em Base64.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjQyLCJleHAiOjE3MzEwMDAwMDB9.4f8s9dK2p...
```

⚠️ **IMPORTANTE:** qualquer um pode ler o payload (é só Base64). **Nunca coloque dados sensíveis aqui** — apenas identificação (userId) e expiração (exp).

Sem a chave secreta, ninguém consegue forjar uma signature válida — por isso o token é confiável.

O payload — dados públicos legíveis

backend/auth.js

```
// Payload simples — só o mínimo necessário
const payload = {
  userId: 42
};
// expiresIn coloca automaticamente a data de expiração
const token = jwt.sign(payload, process.env.JWT_SECRET, {
  expiresIn: "10m"  // expira em 10 min
});
```

Decodificando este token em jwt.io (qualquer um consegue):

```
{
  "userId": 42,
  "iat": 1731000000,
  "exp": 1731000600  ← 10 minutos depois
}
```

Públicos, legíveis, sem nada sensível. O secret (no .env) é quem assina — só o backend consegue forjar.

Validando o token — jwt.verify()

backend/middleware.js

```
// Middleware: toda requisição passa por aqui
export const autenticar = (req, res, next) => {
  const token = req.cookies.token;
  try {
    // Valida assinatura, expiração e decodifica
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // Se chegou aqui: token é válido e não expirou
    req.usuario = { id: decoded.userId };
    next();
  } catch (erro) {
    // Token inválido, expirado ou assinatura ruim
    res.status(401).json({ erro: "Não autenticado" });
  }
};
```

`jwt.verify` faz 3 coisas: **1)** confere a assinatura com o secret, **2)** confere se expirou (exp > agora), **3)** decodifica. Qualquer falha = exceção.

Fluxo completo: bcrypt + JWT

backend/login.js

```
app.post("/login", async (req, res) => {
  const { email, senha } = req.body;
  // 1. Busca o usuário no banco
  const usuario = await db.usuarios.findOne({ email });
  if (!usuario) return res.status(401).json(...);
  // 2. Compara a senha (bcrypt tem salt dentro)
  const ok = await bcrypt.compare(senha, usuario.senhaHash);
  if (!ok) return res.status(401).json(...);
  // 3. Cria JWT simples (só userId, expira em 10min)
  const token = jwt.sign(
    { userId: usuario.id },
    process.env.JWT_SECRET,
    { expiresIn: "10m" }
  );
  // 4. Devolve em cookie HttpOnly
  res.cookie("token", token, { httpOnly: true, maxAge: 600000 });
  res.json({ msg: "Login ok" });
});
```

---

bcrypt protege a senha. **JWT** cria a sessão sem guardar no servidor. **HttpOnly** protege o token de XSS.

JWT dentro de um cookie HttpOnly

A prática mais segura é o backend devolver o JWT dentro de um cookie HttpOnly:

- → **Autenticação** — JWT no cookie é enviado automaticamente em toda requisição, sem frontend montar header manualmente
- → **Expiração** — Max-Age do cookie e `exp` do JWT trabalham juntos
- → **Segurança** — por estar em HttpOnly, o token fica protegido de roubo via `document.cookie` por XSS

Essa combinação — JWT + cookie HttpOnly + expiração — abre a porta para segurança e API contextual.

CORS — o porteiro do navegador

Same-origin policy: é como um **porteiro de clube**. JS rodando em uma página só faz requisições livremente para o mesmo domínio. Se trocar protocolo, domínio OU porta — o porteiro barra.

```
Frontend (Vite)
http://localhost:5173

↓ tenta chamar ↓

Backend (Express)
http://localhost:3000

🚫 Porteiro bloqueia!
(mesma máquina, portas diferentes = origens diferentes)
```

Os headers CORS — crachá VIP

O backend responde com headers especiais — o porteiro recebe as permissões:

response.http

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://localhost:5173
↑ qual frontend pode acessar
Access-Control-Allow-Methods: GET,POST,PUT,DELETE
↑ que tipos de requisição são permitidas
Access-Control-Allow-Headers: Content-Type,Authorization
↑ que headers o frontend pode enviar
Content-Type: application/json
{ "dados": "aqui vai a resposta" }
```

Sem esses headers, resposta chega, mas o porteiro não deixa o JS ler.

Implementando no Express

backend/server.js

```
// middleware CORS — roda ANTES de qualquer rota
app.use((req, res, next) => {
  res.header("Access-Control-Allow-Origin", "http://localhost:5173");
  res.header("Access-Control-Allow-Methods", "GET,POST,PUT,DELETE");
  res.header("Access-Control-Allow-Headers", "Content-Type,Authorization");
  next();
});
// suas rotas vêm DEPOIS desse middleware
app.get("/usuarios", (req, res) => {
  res.json({ usuarios: [{ id: 1, nome: "Ana" }] });
});
app.post("/login", (req, res) => {
  // lógica de autenticação aqui
});
```

O middleware roda em **toda requisição** — frontend recebe os headers, porteiro libera.

Tudo junto agora

O navegador como ambiente: `window` acima, `document` como propriedade, com seus próprios eventos (resize, scroll).

A memória do navegador exposta: Elements (DOM), Console (logs + JS ao vivo), Network (cascata).

Comunicação segura: cookies (Max-Age, HttpOnly, Secure), JWT + cookie HttpOnly, CORS.

Base sólida para construir — agora com proteção e entendimento do próprio ambiente de execução.

Trabalho Final 04

Implementar UPDATE e DELETE no gerenciador de tarefas.

📝 **Acesse:**

https://docs.google.com/forms/d/e/1FAIpQLSfQYQpk55_9VKsyME84-QnLjedeBv_KXPD5RHBAZPxwnPnRjQ/viewform?usp=publish-editor

⏰ **Prazo:** 21/09/2026T23:59:59

Navegador, window, DevTools, cookies, JWT, CORS.

Refinar a comunicação entre frontend e backend, protegendo ambos os lados.